University of Santo Tomas

Institute of Information and Computing Sciences

Department of Information Technology

SOFTWARE ENGINEERING MANUAL

for

Software Engineering 1

Software Engineering 2

Prepared by: Asst. Prof. Mia V. Eleazar

Version 2_2021

Table of Contents

Introduction

According to www.bie.org, **Project Based Learning (PBL)** is "a teaching method in which students gain knowledge and skills by working for an extended period of time to investigate and respond to an authentic, engaging and complex question, problem, or challenge" (What is Project Based Learning (PBL)?, 2018). This course is designed using PBL wherein the focus is on a student-learning approach as students get to acquire knowledge and skills, not just on a classroom setup, but in a real-world environment by identifying and solving real-world problems. Through PBL, students are able to work collaboratively, engaging them to investigate, reflect, and refine on the solutions they have come up with using digital tools. The basis of PBL lies in the authenticity or real-life application of the research being done. The goal is create a learning environment wherein students "learn by doing". (Project Based Learning, 2018)

This Software Engineering Manual was designed using the PBL approach so students can experience tackling real, client-based problems, coming up with solutions, and implementing said solutions in an actual working environment.

Definition

System analysis is the process of identifying an organization's purpose and goal in order to create a process or system which can help the organization achieve them. In IT, the goal is to create automated systems which can help improve business productivity and achieve end-user efficiency. **Systems Analysis and Design (SAD)** deals with the system development activities as mentioned in the Software Development Life Cycle (SDLC). (The Systems Development Life Cycle, 2018)

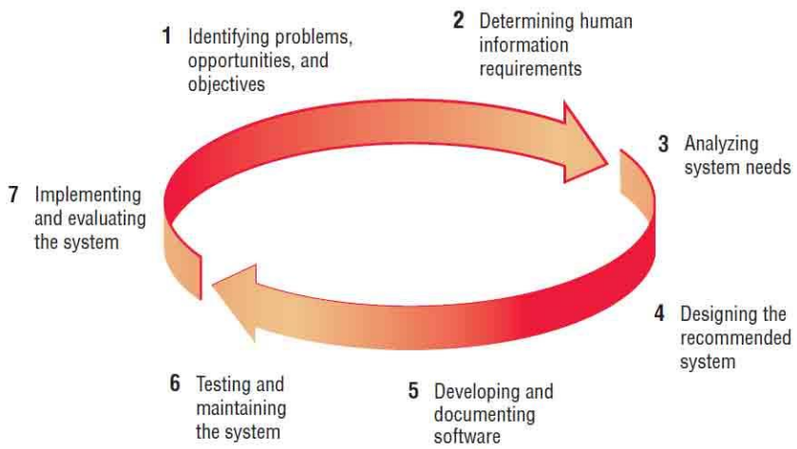


Figure 1: SDLC Phases

Other methods in system development include **Agile Methods**, **Object-Oriented Approach (OOA),** and the basic **Waterfall Method.** The differences among the four – SDLC vs. Agile Vs. OOA vs. Waterfall, are not that different. Overall, the project team needs to understand the organization first. They then create a project proposal identifying requirements and scope through research, observation and actual interviews. The succeeding phases are also similar. The SDLC and OOA both require extensive planning and use of diagrams for the design. The Agile approach and the OOA both allow subsystems to be built incrementally and iteratively until the entire system is completed. The Agile and SDLC approaches are both concerned about the way data is designed and processed throughout the system. (Object Oriented Systems Analysis and Design, 2018)

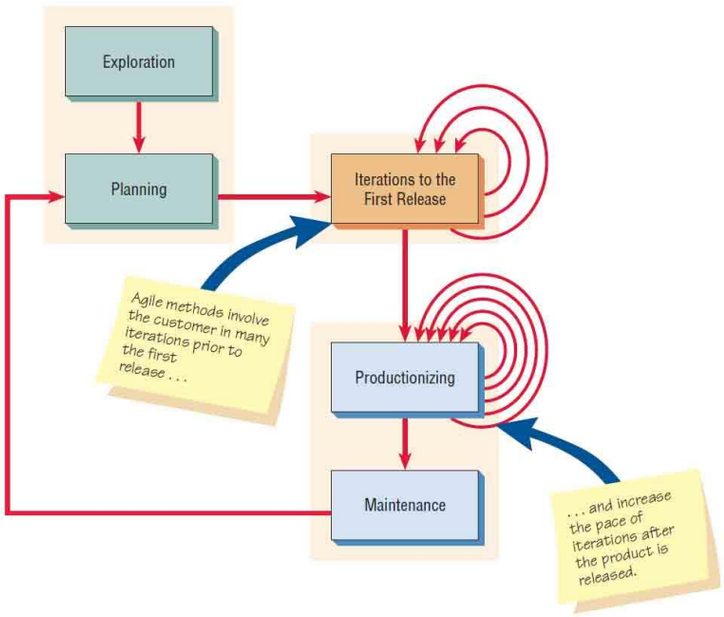


Figure 2: Agile Phases

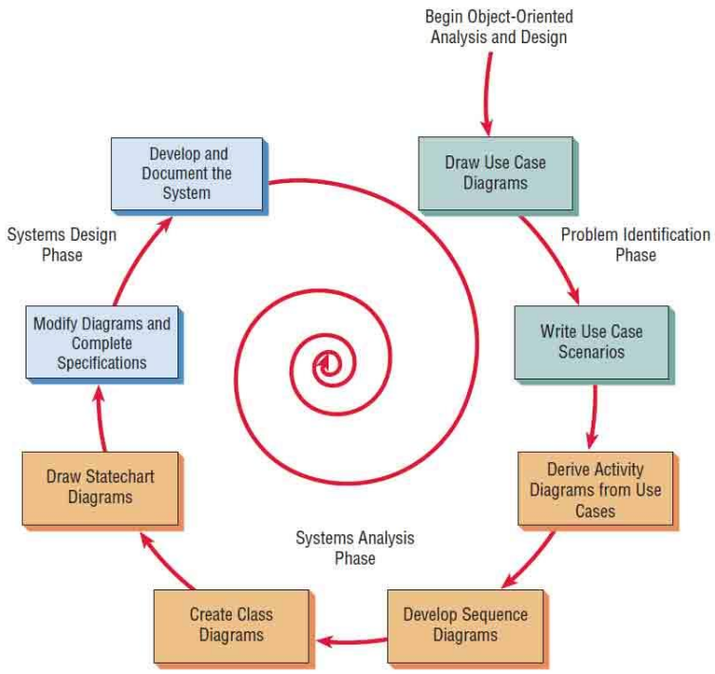


Figure 3: Object Oriented Approach Phases

However, for Software Engineering 1 and 2, as both courses are time-bound (offered on successive semesters) the methodology approach also needs to consider the timetable for said courses. Learning each phase and how each affects the other is an important skillset that students in this course must attain. It is recommended that the Waterfall Model be used in the project development.

The waterfall model uses a linear approach, identifying requirements at the beginning, analyzing the requirements, designing the plan, implementing it by coding, testing and verification, until finally, deploying to end users. The waterfall model is called as such because each phase cascades unto the next one, like water flowing down a waterfall. (The Ultimate Guide.. Waterfall Model, 2021)

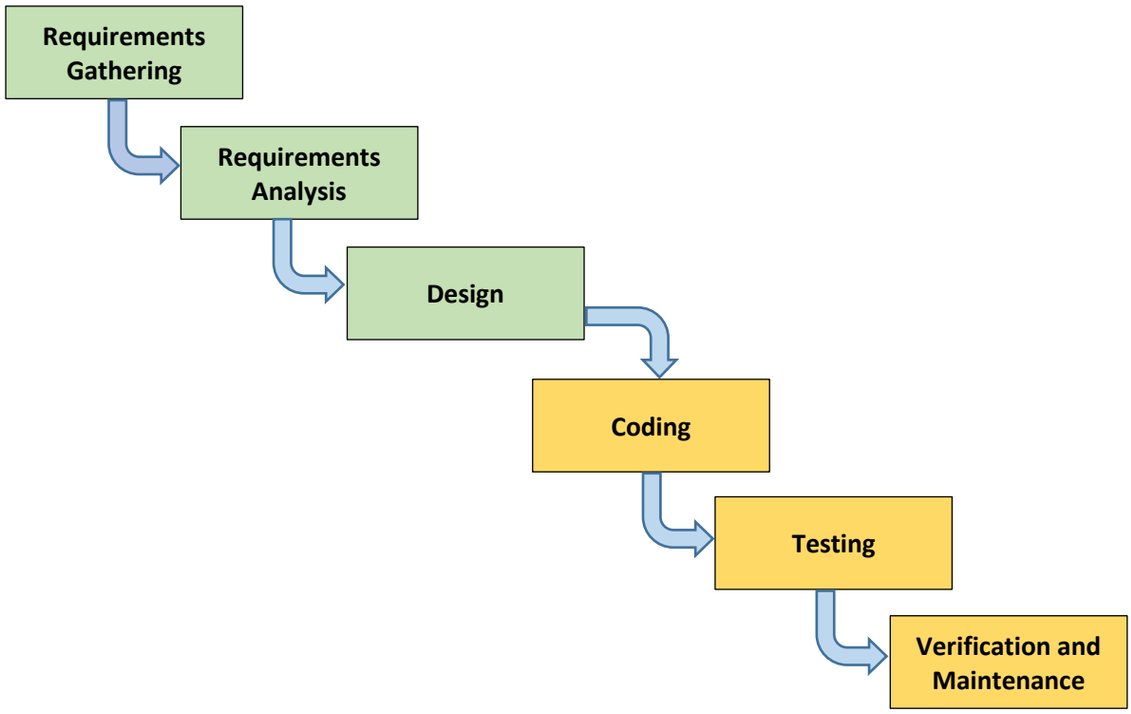


Figure 4: Waterfall Model

Below are the typical phases within a Waterfall Approach along with a short description (The Ultimate Guide.. Waterfall Model, 2021). It is expected that these methodologies will be discussed further in class.

Requirements Gathering: The most important part of this model is that all stakeholder and customer requirements are gathered at the beginning of the project, allowing every other phase to be executed based on the gathered requirements.

Requirements Analysis: Gathered requirements are analyzed to check for consistency and redundancies. This is where an initial project scope is identified. It is assumed that after this phase, there is no need to gather any other requirement from the stakeholders and customers. Water cannot travel upwards!!

Design: The design phase is categorized into two subphases: logical design and physical design. The logical design is when possible solutions are brainstormed and theorized. Based from the project scope, functional and non-functional requirements are identified. The physical design subphase is when those requirements are made into schemas and diagrams that can help visualize the project outcome.

Coding: The coding phase is when programmers analyze the requirements and specifications from the previous phases and produce actual code.

Testing: The testing phase is categorized into two – the alpha testing and the beta testing. Alpha testing is conducted by the system developers to check for errors and bugs prior to verification of end users.

Verification and Maintenance: Verification, also called beta testing, is when the customer uses the system to make sure that it meets the requirements set during Requirements Gathering and Analysis phase. As the customer would be using the system during the maintenance phase, they are expected to discover bugs, inadequate features and other errors that may have been missed during Coding and Testing. The team should apply these fixes as necessary until the customer is satisfied.

Software engineering involves the design and development of a software and how it can be managed. In software engineering, software design and development is implemented, including operation and maintenance. Not to confuse with **Systems Analysis and Design (SAD)** and **Systems Engineering**, Software Engineering deals mainly with software design and development. SAD and Systems Engineering encompass software, hardware, and even working environment implementation. (What is the Difference Between Software Engineering and Software?, 2018)

W3Computing.com has identified several factors in choosing which method would be applicable for certain projects. This can be seen in Table 1.

Table 1: SDLC vs Agile vs Object Oriented Approach

CHOOSE	WHEN
The Systems Development Life Cycle (SDLC) Approach	• systems have been developed and documented using SDLC • it is important to document each step of the way • upper-level management feels more comfortable or safe using SDLC • there are adequate resources and time to complete the full SDLC • communication of how new systems work is important
Agile Methodologies	• there is a project champion of agile methods in the organization applications need to be developed quickly in response to a dynamic environment • a rescue takes place (the system failed and there is no time to figure out what went wrong) • the customer is satisfied with incremental improvements • executives and analysts agree with the principles of agile methodologies
Object-Oriented Methodologies	• the problems modeled lend themselves to classes • an organization supports the UML learning • systems can be added gradually one subsystem at

a time
• reuse of previously written software is a possibility
• it is acceptable to tackle the difficult problems first

For the courses **Software Engineering 1** and **Software Engineering 2**, students are expected to come up with a client-based project whose aim is for them to understand the entire business process in system development:

1. Identify the client's problem
2. Gather client requirements through research and interviews
3. Create a project proposal for review and approval
4. Develop the proposed software
5. Test the software for errors
6. Implement the software in the client's work environment through installation, training and acceptance tests
7. Do maintenance relying on acceptance test results
8. Handover the software once client is satisfied

Projects do not need to be complicated. Hardware, system networks, and system integration are not required. Systems developed will be based on client requirements and specifications. The goal for both courses is to prepare students for their Thesis and Capstone Projects during their final year as requirements and specifications are very much similar, if not the same (i.e. Capstone Project for BS IT). As such, students are expected to learn the following:

1. The basics in business processes
2. Project requirements gathering
3. Project management and design
4. Preparing technical documents
5. Presenting in front of a technical panel

Course Requirements and Schedule

Software Engineering 1 is taken during the 1st semester of the students' 3rd year for BS IT and BS CS programs. BS IS students take SE1 during the 2nd semester of their 2nd year. Students are expected to have passed **Database Design** and **Web Programming** at a minimum. All programming courses are expected to be recalled and reviewed by the students as they will be used as reference for the system's design and development.

	Year	Semester	Pre-requisite
BS Computer Science	3rd	1st	ICS2607
BS Information Systems	3rd	1st	IS 2636, PURPCOM
BS Information Technology	3rd	1st	PURPCOM

Software Engineering 2 is taken on the subsequent semester after SE1. Course pre-requisite is SE1 (ICS 26010).

	Year	Semester	Pre-requisite
BS Computer Science	3rd	2nd	ICS26010, ICS26011
BS Information Systems	3rd	2nd	ICS26010

BS Information Technology	3rd	2nd	ICS26010

Group Composition

The project is done in groups, with maximum of 8 students, where each one is assigned roles that are often identified in system development within industries. Below is a list of the roles to be assigned.

1. Project Manager
2. System Analyst
3. Business Analyst
4. Project Developer
5. Quality Assurance Officer

Normally, 1 person is assigned per designation, but it is important that the group have at least, 2 developers and 2 QAs during SE2. The role description are as follows:

A **Project Manager** has the responsibility of planning, monitoring, and implementing a project. They are normally the point person for identified stakeholders, such as the client, and must be capable of handling the rest of the project team. Project Managers may not necessarily be directly involved in the activities that produce the end result, such as actual coding, but they must be able to maintain the progress throughout the entire system lifecycle.

The Project Manager should also have good communication skills as they need to interact with the client for progress reports. Good leadership skills are also important as the heads of the different departments within the project report to them. It is also their responsibility to ensure that the project is delivered on time and within the budget. They can do so by identifying ways to reduce the risk of overall failure, maximizing benefits, and minimizing costs. (Project Manager, 2018)

A **Systems Analyst** specializes in analyzing, designing, and implementing systems. They often are in charge of communicating with end users and developers/programmers. It is recommended that Systems Analysts be proficient in IT as they need to be familiar with different programming languages, hardware and software tools, and other technological trends. They decide what languages and technology will be used in developing the system. Systems Analysts also decide on the design methodology to be used. A systems analyst is often assigned to one project and works alongside with a business analyst. These roles, although having some overlap, may not necessarily be the same. (System Analyst, 2018)

A **Business Analyst** is responsible for understanding the business model, business process, and business domain of the client. They are the ones who define how technology can be integrated into the current business process of the company. The Business Analyst works together with the System Analyst to come up with technological solutions, such as tools or applications for the client. A Business Analyst is also involved in the integration, testing, and implementation of the system as they need to analyze end user responses. This would often include creation of training manuals and participating during implementation and maintenance stages.

Both Systems Analyst and Business Analyst are in charge of developing project plans, strategic plans, and other technical and non-technical documentation. In some cases, an individual gets to have training in both fields of expertise and ends up being assigned as both a Systems and Business Analyst in a single project. (Business Analyst, 2018)

Project Developers or simply **Developers**, are familiar with the software development process, including research, design, coding, and testing. Not to be confused with a Programmer, Developers have more in-depth knowledge in systems development, whereas Programmers stick to coding only. (Software Developer, 2018)

Quality Assurance Officers, or **QAs**, are also Developers, as they are also familiar with the software development process, know how to do design and coding. However, it is important to note that QAs test systems for system errors, end user experience, and adherence of the system to business processes.

QAs must also be good in systems checking, documenting any issues found, and finally, resolving said issues. (What does a QA Analyst Do?, 2017) Developers, on the other hand, only check for bugs in their developed code. Which is why the job a Developer is kept separate from a QA. This is to avoid bias and promote system integrity. Having an impartial judgement on the system's performance creates a better quality system.

Below is a table identifying specific tasks for each group member for SE1 up to SE2:

DESIGNATION	No. of staff	TASKS (SE1)	TASKS (SE2)
PROJECT MANAGER	1	identification of all project tasks	Monitoring of all project tasks
		Creation of group's time table; creates project schedule through the use of Gantt Chart to show tasks, task dependencies and critical tasks	Monitoring of group's timetable and making sure all deliverables are submitted on time
		creation of project feasibility plans	Monitoring feasibility plans to ensure they are still in line with actual project outcomes
		Monitors weekly progress report	Monitors weekly progress report via Trello
		leads the team in meeting with client for requirements gathering	Continually updates client of project progress
		Continually updates client of project progress	May take part in the Development team (i.e. help in coding) OR QA team (i.e. help in testing), but not both
		Leads the creation of the Validation Board	Must work closely with QA team in **delivering** the Survey Questionnaires (UAT) to the client and users
		leads the creation of the SPMP Document	

SYSTEMS ANALYST	1	leads the creation of the SDD Document	leads creation of an Installation Manual (if applicable)
		identify system requirements (functional and non-functional), technical requirements, and system/software scope and limitations	Monitor if Dev team and QA team are following the Design model identified in the SDD document
		leads in the design plan, i.e. what methodology to use, what programming language to use, etc.	May take part in the Development team (i.e. help in coding) OR QA team (i.e. help in testing), but not both
		identifies possible risks which may occur during system development	Must work closely with QA team in **writing** the Survey Questionnaires for UAT
		identify operational requirements and other design considerations (i.e. operating systems, software platforms, etc.)	Monitor and fix critical issues which may occur during development phase

		work alongside the Project Manager in creating feasibility plans	
		work alongside QA team to develop the test plans	
		work alongside the Business Analyst to identify the project's technical specifications and create technical diagrams for developers to follow (i.e. class diagram, sequence diagram)	
		considers scalability to ensure that the system can support future growth and expansion	

BUSINESS ANALYST	1	leads the creation of the SRS document	leads creation of a User's Manual (if applicable)
		identifies the business-related requirements for the proposed project by reviewing existing business process flows, forms, documents, and including organizational culture	Continually updates client of project progress
		work alongside the System Analyst to ensure system design meets the company's strategic goals	review project's output (i.e. forms) to check if user expectations/needs are met
		be able to identify system inputs and format outputs to meet user needs	monitor and fix critical issues which may occur during implementation stage (strategic goals must still be met)
		identify possible risks which may occur during system implementation	May take part in the Development team (i.e. help in coding) OR QA team (i.e. help in testing), but not both
		create specifications, flowcharts and model diagrams for developers to follow (i.e. use case, activity diagram)	Must work closely with QA team in **delivering** the Survey Questionnaires (UAT) to the client and users
		should work hand-in-hand with the Project Manager in interviewing the client and other stakeholders	

DEVELOPER	2	creates system prototype (mockups)	creation of the code for the front end (user interface) as per the Design document in SE1

		follow design model created by the System Analyst to create the prototype	creation of the code for the back end (database) as per the Design document in SE1
		must NOT work with QA team in creating test plans	does debugging and unit testing
			continuously sends to QA finished codes/modules for review
			revise reviewed codes/modules identified by QA with errors
			revise codes/modules as per UAT report (maintenance phase) from QA team
			must NOT work with QA team in testing
			handle user training alongside QA team

QUALITY ASSURANCE OFFICER	2	leads in the creation of the Software Test Plan (STP) document	leads in the creation of the Software Test Document
		follow design model created by the System Analyst and requirements model created by the Business Analyst to create the test plans	does unit testing and integration testing
		identifying test plans for all functional and non-functional requirements	examine systems performance issues taken from codes created by the developers
		must NOT work with Developer team in creating prototype	re-test revised codes/modules from developers
			make sure all testing (and re-testing) is documented/recorded
			Leads in the creation of the Survey Questionnaire for UAT; leads in the delivery of the UAT to client and users
			Collect, record, tally (use statistical treatments) and analyze the results of the UAT for project maintenance
			handle user training alongside QA team
			must NOT work with Dev team in coding

Adviser/Panel Members/Client

The **Adviser** of the groups in Software Engineering will be the current Coordinator handling the class section. Any written communication that needs to be submitted to the client must first be approved (signed) by the Adviser. This includes request for interviews, letter of invitation for defense, etc.

In case the Adviser in SE1 and SE2 will differ, it is expected that the SE1 Adviser will need to do a proper turnover to the SE2 Adviser, which could include introduction to project Clients.

The **Panel Members** are invited faculty members within the Institute of Information and Computing Sciences. There will be a minimum of two (2) panelists and a maximum of three (3) during the proposal defense (SE1) and project defense (SE2). It is important that at least one (1) panelist be retained during the project defense (SE2) from the proposal defense (SE1) to avoid inconsistencies. The Adviser can serve as member of the Panel.

The **Client** is someone who is considered to be a decision-maker in their organization or company. Examples would be Upper Management officials, Business Owners, etc. Clients will not be grading the students during the defense, however, they are required to be present during the defense, or at least their representative, for project validation purposes.

Schedule/Time table

Below is the proposed timetable for SE1 and SE2. Note that Prelim Exam and Final Exam weeks have been omitted.

SE1 – Software Engineering 1

Week 1	Create a client list
Week 2	Find a client Finalize client
Week 3	Requirements gathering: Conduct interviews Create Validation Board
Week 4	Requirements gathering: Conduct interviews Create Validation Board
Week 5	Requirements gathering: Conduct interviews Create Validation Board
Week 6	Requirements gathering: Conduct interviews Finalize Validation Board
Week 7	Present Validation Board
Week 8	Finalize Software Project Management Plan and Software Requirements Specification
Week 9-11	System Design
Week 12	System Design Create Test Plan
Week 13	Finalize Software Design Document and Software Test Plan
Week 14-15	Proposal Defense

Week 16	Revisions

SE2 – Software Engineering 2

Week 1	Presentation of Accepted Project Documents Coding
Week 2-5	Coding and Testing
Week 6	Finalize Software Design Document and Software Test Document
Week 7	Project Defense
Week 8	Project Revisions
Week 9	Project Deployment: Installation and Training
Week 10-12	Project Deployment: End User
Week 13-14	Project Deployment: End User User Acceptance Testing
Week 15-16	Software Maintenance
Week 17	Software Maintenance Final Deliverables submission

Project Proposal

- Set during Software Engineering 1 (SE1)
- Due to the requirements and guidelines set by the CHED memorandum on off-campus activities, it is **highly recommended** that all identified clients for Software Engineering 1 should be within UST campus only. This includes administrative offices, academic offices, and student services. Students and student organizations, however, cannot serve as clients for a proposed project relating to student services. In lieu of it, Org Advisers, Academic Officials, or preferably, Administrative Officials, such as the SWDB Coordinator, may serve as the "client" for projects relating to student services.
- In case the group has identified a Client that is off-campus will only be accepted once the project team submits the following **not later than Week 3**:
    - Company details (Name of owner, business address, contact details, business model/company org chart)
    - Letter of acceptance from company (in company letterhead) signifying their acceptance of your intent to create a project for them. Letter must be addressed to the Adviser. The letter need not be specific on the project already, but at least, an acceptance of your intent to create one.
    - Signed Letter of non-relation. *See Appendix G for Letter Formats.*
    - Individual Medical Certificates for all team members from a DOH licensed clinic or hospital
    - Parental Consent (kindly see IICS SWDB Coordinator for the form)
- Each group must write a Proposal Letter to their identified clients to formalize the intent (*See Appendix G for Letter formats*). A received copy must be submitted to the Adviser for validation.
- For a proposal to be accepted, the panel members must deem your project "Ready for Development" after the proposal defense set during the final term period (*See Appendix H for Evaluation Sheets*).

Project Deployment

- Set during Software Engineering 2
- At the start of the term, Advisers must check if the SE1 project has been approved for development. Groups must submit their signed Project Development form (*See Appendix I for Forms*) as well as a signed cover page of all SE1 documents, signifying revisions have been checked. Signatories on the cover page should include all Panel Members, Client, and Adviser. Documents must be submitted not later than Week 2 of SE2.
- Final project defense is set before the Prelim Exam Week or immediately after to give groups enough time for the project Deployment. A Project Deployment Form must be signed and submitted to the Adviser prior to Client deployment which is broken down as follows:
    - System installation and end-user training – 3-4 days
    - Deployment and use – 3-4 weeks
    - User acceptance testing – 1-2 weeks
    - Maintenance – 1-2 weeks

  This is following industry standards on a much smaller scale, simulating actual practices. It is highly recommended that groups follow the said timeline to achieve reliable final reports.

- Groups who fail to follow the scheduled defense prior to the Prelim Exam week will be given deductions on their final defense grade

Online Defense Guidelines

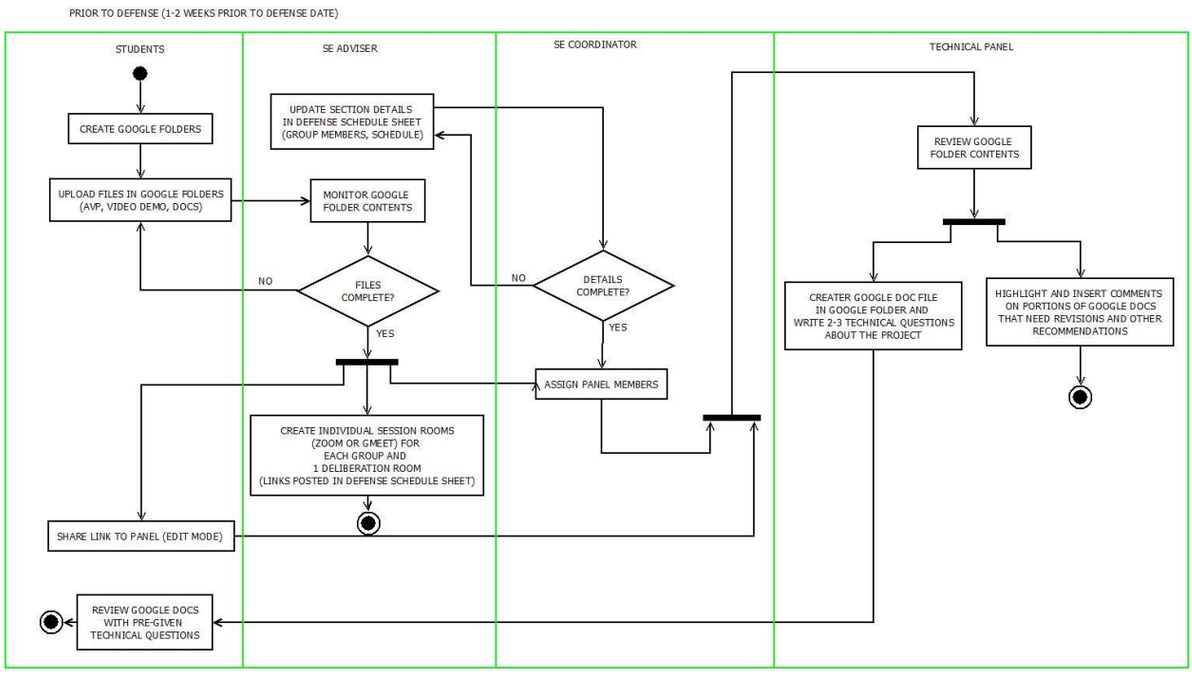
In preparation for the online oral defense, each group must create a Google Folder, in which all group members, Adviser, and Panel members shall have EDIT access. This folder is created during SE1 and updated with additional files and documents up to SE2.

The Google Folder must have the following sub-folders and their content:

- Document – where working Google document is saved, like a draft
- Presentation – where all files needed for defense is saved, including but not limited to:
    - AVPs (not exceeding 45 min.) – for SE1 and SE2
    - Video demos (with audio narrative) – for SE2
    - Final Google document to be accessed by panel members – for SE1 and SE2
- Appendices – supporting files (pictures, video recordings, etc) and other pertinent documents

Before the oral defense

The Adviser identifies if the groups in their respective class are ready or not. Their final list is then submitted to the Course Coordinator at least two (2) weeks prior to the defense dates. The defense schedule is prepared by the Course Coordinator and shared to all Advisers at least five (5) working days prior to the defense dates.



During the oral defense

To keep track of all comments during the defense, the online session will be recorded after getting the consent of the group.

All group members must be present during the defense. Any questions or clarifications that the Client may wish to discuss during this time may be brought up during the Deliberation only.

In case groups are a no-show without prior notice, they are to be given a grade of zero (0) for their defense grade, until such a time their Adviser deems them acceptable for a re-defense. If a member is a no-show without prior notice, said member will automatically get a grade of zero (0) for their individual grade.

Group members must be in proper business attire during the oral defense as can be seen on camera:

Men: Long sleeve shirts with necktie
Formal Coat (recommended)

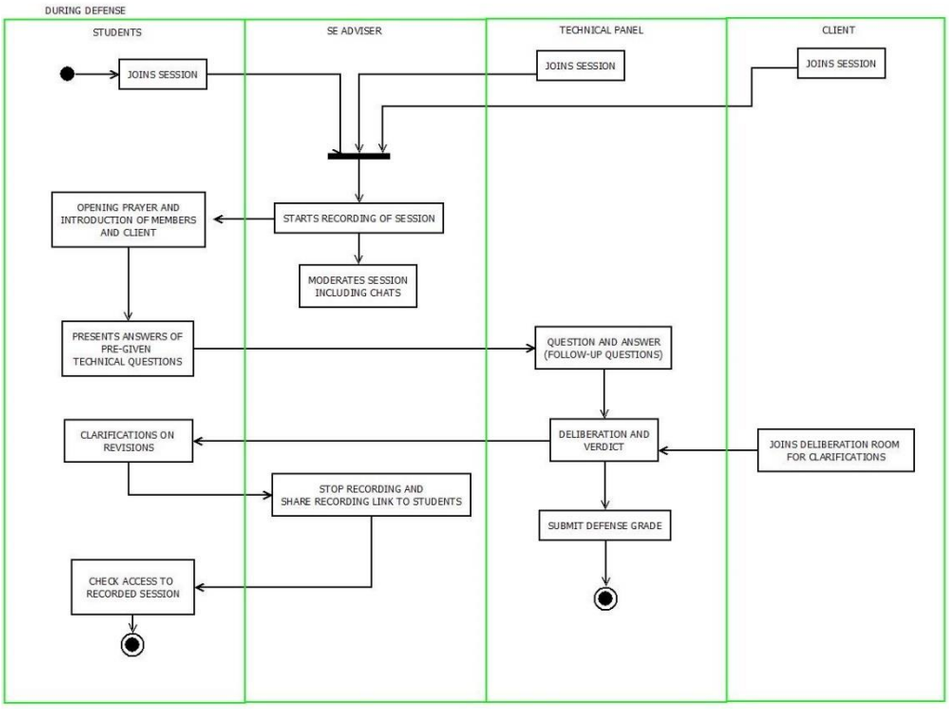
Ladies: Dress shirts/blouses (no sleeveless tops)
Blazers (recommended)

Group members must use the official ust.edu.ph email in accessing the session room so their names are on display for easier identification.

The duration of the online oral defense is fifty (50) to sixty (60) minutes, broken down as follows:

SE1:	Introduction	5 min.
	Answering of pre-defense questions	10 min.
	Q&A follow up	20 min.
	Panel deliberation	10 min.
	Announcement of verdict	5 min.

SE2:	Introduction	5 min.
	Answering of pre-defense questions	15 min.
	Q&A follow up	25 min.
	Panel deliberation	10 min.
	Announcement of verdict	5 min.



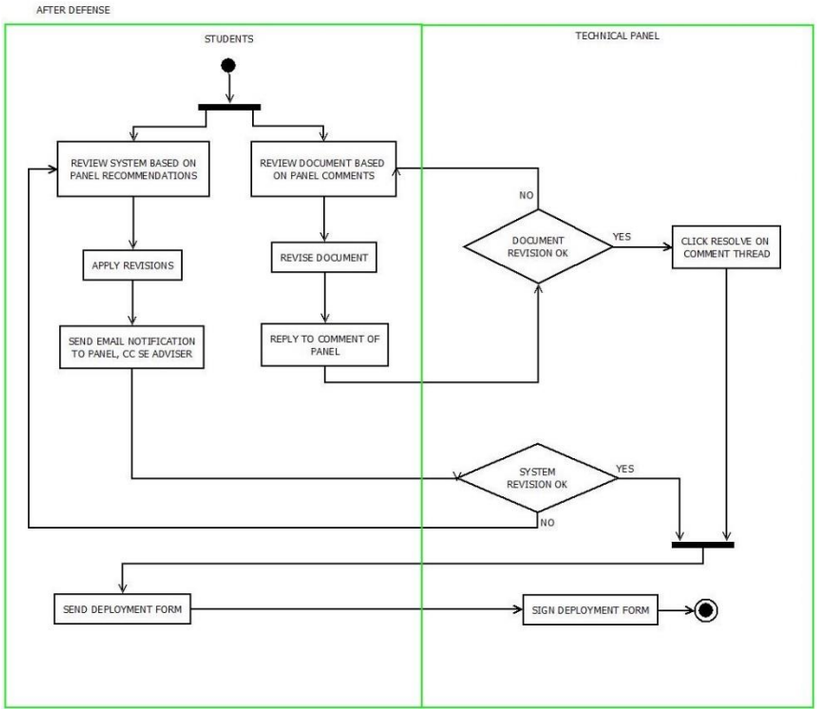
After the oral defense

Panel revisions and recommendations are to be reviewed and revised by the students as their final requirement for the course. Groups are given 1-2 weeks for document revision in which panel members must resolve all comments on their documents.

Once all comments are resolved, each group must submit the following final deliverables:

SE1: revised SPMP in PDF
revised SRS in PDF
revised SDD in PDF
revised STP in PDF
Signed Development Form (See Appendix I)

SE2: revised SPMP in PDF
revised SRS in PDF
revised SDD in PDF
revised STP in PDF
Signed Deployment Form (See Appendix I)



Project Document

The documents that need to be prepared are technical in nature. For this reason, the IEEE standard for Software Development Process was used as a basis in our Software Engineering documents. However, as the IEEE standards are industry-grade and very detailed-oriented to be used as introduction to system development, the content has been reduced to follow the outcomes set by both courses. Below is the list of documents required for both SE1 and SE2. The original IEEE formats have been cited and links can be found at the References page of this manual.

Software Engineering 1 (SE1) has four (4) documents, namely:

1. **Software Project Management Plan** (IEEE Standard for Software Project Management Plans)
   The Software Project Management Plan (SPMP) should include client information, current business/process model/s, risk analysis, and feasibility studies. This document will verify if there is an actual need for the project and also determine if the project is viable to everyone involved. *See Appendix A for SPMP content.*

2. **Software Requirements Specification** (IEEE Software Requirements Specification Template)
   The Software Requirements Specification (SRS) should include project scope, descriptions, and other system features. *See Appendix B for SRS content.*

3. **Software Design Document** (Software Design Document Template)
   The Software Design Document (SDD) should include the design of the system including the system architecture, data design, and component design. Majority of the content will include design figures using UML. *See Appendix C for SDD content.*

4. **Software Test Plan** (Test Plan Template (IEEE 829-1998 Format))
   The Software Test Plan includes a list of the system's items that need to be tested and the approach to be used for testing them. As this document is only a plan, content will focus on identifying testing features and what criteria to use for each feature. *See Appendix D for STP content.*

Software Engineering 2 (SE2) has four (4) documents, 2 which, are to be retained from SE1:

1. **Software Project Management Plan** (IEEE Standard for Software Project Management Plans)
   The SPMP submitted must be the same as the one approved in SE1.

2. **Software Requirements Specification** (IEEE Software Requirements Specification Template)
   The SPMP submitted must be the same as the one approved in SE1

3. **Software Design Document** (Software Design Document Template)
   The original document submitted in SE1 can undergo revisions based on the identified issues of the group during the Development phase.

4. **Software Test Document** (Test Plan Template (IEEE 829-1998 Format))
   This document is derived from the Software Test Plan in SE1 but with test cases included (Test Case Specification Template (IEEE 829-1998)). As the IEEE format is extensive, the test case format to be used will be a generic one, but with all pertinent details still intact. (Sample Test Case Template with Test Case Examples, 2018). *See Appendix E for Test Case Template.*

IPR and Project Ownership

All Software Engineering projects must be developed personally by the group members and must not infringe on any existing intellectual property. Proper citation must be given to previous projects, including but not limited to open source software and technology.

If a project has been deemed to infringe on the Intellectual Property Rights of other existing projects (including open source software and technology), without giving proper citations, the team members and their project will be put under investigation by a committee headed by the Department Chair and at least two (2) faculty members, not including the Panel Members. Students who are found to have violated said guideline will be subject to disciplinary action by the IICS' SWDB and will have a pending grade of INC until the matter has been resolved. Worst case scenario would be a group may be asked to propose a new project altogether.

The system, including codes and documentation, will be handed over to the client upon finishing SE2. A **Sign-Off Sheet** will be used for a formal turnover to signify transfer of ownership to the client *(See Appendix F for List of Final Project Deliverables)*. However, copies of the code and document will be retained by the Adviser for academic purposes only.

All information from the client and end-users are to be used for research and academic purposes only. Part of the final project requirements is the submission of actual **System Reports**. Any information that the Client deems confidential in the submitted reports may be redacted upon their discretion without compromising the validity of the said report *(See Appendix F for List of Final Project Deliverables)*.

Grading system

As the course was designed to use a PBL approach, it is recommended that the grading system be as follows:

SE1 – Software Engineering 1

- Individual Long Quizzes – 15%
- Group Activities – 10%
- Peer evaluation – 10%
- Alternative assessments:

- Prelim exam – Validation board + group presentation – 25%
- Final exam – Project proposal defense and documents – 30%

SE2 – Software Engineering 2

- Individual Long Quizzes – 15%
- Group Activities – 20%
- Peer evaluation – 10%
- Alternative assessments – 25%
  - Prelim exam – Final project defense, prototype, project documents
- Final exam – cover to cover – 20%
- Project reports – 10%

Submission of Requirements

Project requirements are defined by the Adviser based on the course syllabus. Major project requirements for SE1 include:

1. Validation board – prelim period (*See Appendix J for a copy of the Validation Board template*)
2. Software project management plan – prelim period
3. Software requirements specification – prelim period
4. Software design document – final period
5. Software test plan – final period

Major project requirements for SE2 include the following:

1. Revised software design document – prelim period
2. Software test document – prelim period
3. User acceptance test (survey) questionnaire – prelim period
4. Final project deliverables – final period (*See Appendix F for List of Final Project Deliverables*)

APPENDIX A
Software Project Management Plan Document

1. Introduction
This section of the SPMP provides an overview of the project.

1.1 Project Overview
Include a concise summary of the project objectives, major work activities, major milestones, required resources, and budget. Describe the relationship of this project to other projects, if appropriate. Provide a reference to the official statement of product requirements.

1.2 Project Deliverables
List the primary deliverables for the customer, the delivery dates, delivery locations, and quantities required satisfying the terms of the project agreement.

2. Project Organization
This section specifies the process model for the project and its organizational structure.

2.1 Process Model
Specify the life cycle model to be used for this project or refer to an organizational standard model that will be followed. The process model must include roles, activities, entry criteria and exit criteria for project initiation, product development, product release, and project termination.
Figure P-1: Project Process Model

2.2 Organizational Structure
Describe the internal management structure of the project, as well as how the project relates to the rest of the organization. It is recommended that charts be used to show the lines of authority.
Figure P-2: Organization Chart

2.3 Project Responsibilities
Identify and state the nature of each major project function and activity, and identify the individuals who are responsible for those functions and activities. Tables of functions and activities may be used to depict project responsibilities.

i.e.:
Role Description Person
Project Manager leads project team; <name> responsible for project deliverables
Technical Team Leader(s) <define as locally used> <name>
<etc.> <etc.>

Table P-1: Project Responsibilities

3. Managerial Process
This section of the SPMP specifies the management process for this project.

3.1 Management Objectives and Priorities
Describe the philosophy, goals, and priorities for managing this project. A validation board might be helpful in communicating what dimensions of the project are fixed, constrained and flexible.
Figure P-3: Validation Board

3.2 Assumptions, Dependencies, and Constraints
State the assumptions on which the project is based, any external events the project is dependent upon, and the constraints under which the project is to be conducted. Include an explicit statement of the relative priorities among meeting functionality, schedule, and budget for this project.

Specific reports would include operational feasibility, technical feasibility, and economic feasibility. This may be documented in this part or included as part of the appendices.

3.3 Risk Management

Describe the process to be used to identify, analyze, and manage the risk factors associated with the project. Describe mechanisms for tracking the various risk factors and implementing contingency plans. Risk factors that should be considered include contractual risks, technological risks, risks due to size and complexity of the product, risks in personnel acquisition and retention, and risks in achieving customer acceptance of the product.

The specific risks for this project and the methods for managing them may be documented here or in another document included as an appendix or by reference.

3.4 Monitoring and Controlling Mechanisms

Define the reporting mechanisms, report formats, review and audit mechanisms, and other tools and techniques to be used in monitoring and controlling adherence to the SPMP. Project monitoring should occur at the level of work packages. Include monitoring and controlling mechanisms for the project support functions (quality assurance, configuration management, documentation and training).

> **Trello** will be used to show the reporting and communication plan for the project. The communication table can show the regular reports and communication expected of the project, such as weekly status reports, regular reviews, or as-needed communication. The exact types of communication vary between groups, but it is useful to identify the planned means at the start of the project.

Table P-2: Communication and Reporting Plan

4. Work Packages, Schedule, and Budget

Specify the work packages, dependency relationships, resource requirements, allocation of budget and resources to work packages, and a project schedule. Much of the content may be in appendices that are living documents, updated as the work proceeds.

4.1 Work Packages

Specify the work packages for the activities and tasks that must be completed in order to satisfy the project agreement. Each work package is uniquely identified. A diagram depicting the breakdown of project activities and tasks (a work breakdown structure) may be used to depict hierarchical relationships among work packages.

Table P-3: Work Breakdown Schedule

4.2 Dependencies

Specify the ordering relations among work packages to account for interdependencies among them and dependencies on external events.

Techniques such as dependency lists, activity networks, and the critical path method may be used to depict dependencies among work packages.

Figure P-4: Dependency Diagrams

4.3 Resource Requirements

Provide, as a function of time, estimates of the total resources required to complete the project. Numbers and types of personnel, computer time, support software, computer hardware, office and laboratory facilities, travel, and maintenance requirements for the project resources are typical resources that should be specified.

4.4 Resource Allocation

Specify the allocation of resources to the various project functions, activities, and tasks.

Table P-4: Resource Allocation

4.5 Schedule

Provide the schedule for the various project functions, activities, and tasks, taking into account the precedence relations and the required milestone dates. Schedules may be expressed in absolute calendar time or in increments relative to a key project milestone.

Table P-5: Gantt Chart

5. Additional Components

Certain additional components may be required and may be appended as additional sections or subsections to the SPMP. Additional items of importance on any particular project may include subcontractor management plans, security plans, independent verification and validation plans, training plans, hardware procurement plans, facilities plans, installation plans, data conversion plans, system transition plans, or the product maintenance plan.

5.1 Appendices

Appendices may be included, either directly or by reference, to provide supporting details that could detract from the SPMP if included in the body of the SPMP. Suggested appendices include:

A. Current Top 10 Risk Chart
B. Current Project Work Breakdown Structure
C. Current Detailed Project Schedule
D. Operational Feasibility
E. Technical Feasibility
F. Economic Feasibility

APPENDIX B
Software Requirements Specifications

1. Introduction
   This documents the purpose and scope of the project

   1.1. Project Purpose
   Define why there is a need for the project. A fishbone analysis may be helpful in identifying project purpose. Identify project's beneficiaries and stakeholders. Introduce the individuals whom the group interviewed and a summary of the discussion. Full client interview transcripts must be included in the appendices.
   Figure R-1: Fishbone diagram

   1.2. Project Scope
   Describe what the project can do. Identify project goals and objectives, making sure they are measurable and attainable based on project plan. A Validation Board will help identify the project's scope and limitations.
   Figure R-2: Javelin Board

2. Overall description
   This documents what the project can do and its requirements

   2.1. Project perspective
   Create a model of the system by identifying who are the system's actors, what are its primary functionalities and capabilities
   Figure R-3: Use case diagram

   2.2. User accounts and characteristics
   Identify the users of the system and their characteristics. Describe how each one will interact with the system. Actor interaction must also be identified, if any.

   2.3. Project functions
   Create a model of the system process flow from start to end, including decision trees and generation of reports, if any.
   Figure R-5: Activity diagram

   2.4. Operating environment
   List and describe the system's functional requirements by identifying what the project is capable of doing. Non-functional requirements are those which needed to validate the project's performance. Non-functional requirements may include system reliability, maintainability, efficiency, security and usability. Identifying non-functional requirements often entail the need for user acceptance tests (UAT) which would be later identified in the System Test Plan (STP) document
   Table R-1: Functional requirements
   Table R-2: Non-functional requirements

   2.5. Design and implementation constraints
   This documents the limitations, dependencies and possible consequences on the system's design, development and implementation

      2.5.1. Risk assessment
      Describe risks involved during system design, development and implementation.

      2.5.2. Assumptions and dependencies

Describe how the system may affect organization productivity and processes.
Identify factors which may affect the organization upon use of the system.

2.6. User documentation
Identify the need for user training and documentation such as user manuals and installation manuals. Describe how these will be implemented and what would be the expected outcomes.

3. External Interface Requirements
   3.1. User interfaces
   Describe how users will interact with your proposed system via the system's UI.

   3.2. Hardware interfaces (if any)
   Describe how users will interact with any hardware peripherals used in the proposed system, if any. Also include how the hardware peripheral will interact or connect with your proposed system.
   Figure R-6: System block diagram

   3.3. Software interfaces (if any)
   In case the organization already has an existing system and your proposed system will need to be integrated to it, describe how the interaction or connection will occur.
   Figure R-7: System integration diagram

   3.4. Communication interfaces
   Describe how the system will be implemented, if online, offline or stand-alone. In case the system is online, what would be the requirements needed to access it.

4. System features
   List down and explain each system requirement, focusing on system's functional requirements.
   4.1. Requirement 1
   4.2. Requirement 2
   4.3. …

5. Business rules
   Identify who will interact with the system and possible dependencies to and from other offices in the organization.

6. Suggested Appendices
   6.1. Interview transcripts

APPENDIX C
Software Design Document

1. System Overview

   Give a general description of the context (client, background, objectives, etc.), functionality (functional requirements), and design (external interface requirements) of your project. Provide any background information if necessary.

2. System Architecture
   a. Architectural design

      Describe how responsibilities of the system were partitioned and then assigned to subsystems. Describe how each subsystems collaborate with one another to achieve the desired functionality. This section should be able to give a general understanding of how and why the system was decomposed and how each subsystem work together. Diagrams must include brief narratives to support it.

      Figure D2.1. Detailed Use case diagram (with dependencies, inclusions, etc.)

      Figure D2.2. Swimlane diagram

   b. Decomposition Description

      This section provides the in-depth decomposition as mentioned in the architectural design. Diagrams should identify system sequence, subsystem model, and interface specifications. Depending on the number of subsystems, the Subsystem Sequence Diagrams must be shown separately from the general System Sequence Diagram.

      Figure D2.3. System sequence diagram

      Figure D2.4.1. Subsystem sequence diagram: Function 1

      Figure D2.4.2. Subsystem sequence diagram: Function 2

      ….

      Figure D2.4.x. Subsystem sequence diagram: Function x

3. Data Design
   a. Data Description

      Explain how the information domain of your system is transformed into data structures. Describe how the major data or system entities are stored, processed and organized. List any databases or data storage items.

      Figure D3: E-R diagram

   b. Data Dictionary
      Alphabetically list the system entities or major data along with their types and descriptions. Include attributes, methods and method parameters.

      Table 3: Data dictionary

4. Component Design
   Summarize each object member function for all the objects. Include local data, if any. Diagrams must include brief narratives to support it.

Figure D4: Class diagram

5. Human Interface Design *(for SE1)*
   5.1. Overview of User Interface
   Describe the functionality of the system from the user's perspective. Explain how the user will be able to use your system to complete all the expected features and the feedback information that will be displayed for the user.

   5.2. Screen Mockups
   Display screenshots showing the interface from the user's perspective. Use an automated drawing tool, such as Balsamiq Mockups, to create user interface screens. Take note that these designs are expected to be fully functional and working only in SE2 (Software Engineering 2). While partial functionality may be good, it is not a requirement for SE1 (Software Engineering 1).

6. Human Interface Design *(for SE2)*
   6.1. Overview of User Interface
   Describe the functionality of the system from the user's perspective. Explain how the user will be able to use your system to complete all the expected features and the feedback information that will be displayed for the user.

   6.2. Screenshot of system
   Display screenshots showing the interface from the user's perspective. Show all major functionalities.

APPENDIX D
Software Test Plan

1. Introduction
   State the purpose of the Software Test Plan by identifying the scope of the testing effort, how testing relates to other evaluation activities (analysis and review of UAT), and possible process to be used for maintenance and/or change control.

   1.1. Test Items
   These are the things you intend to test within the scope of this test plan. This list can be identified from your SRS and SDD documents. Remember, what you are testing is what you intend to deliver to your client. Start identifying from higher levels, such as application or functional areas (i.e. login success, report generation) down to lower levels such as programs, units, modules or builds (i.e. password encryption, computation accuracy, data handling)

   1.2. Test Approach
   This is your overall test strategy for this test plan and should be appropriate to the level of the plan. Your test plan must include a Master Plan and an Acceptance Plan.

   Often, a Master Plan's test approach matches the methodology identified in developing the system as mentioned in the SPMP. Agile methods in testing would require iterations and incremental testing of a modular approach. You need to describe how you would plan to do this. SDLC methods would normally use a functional approach in testing. This you would also need to elaborate further based on your system requirements. Coverage requirements must also be identified. Lower levels under the Master Plan may include the following:
   a. Unit testing (black box testing)
   b. Integration testing (white box testing)
   c. User acceptance testing (UAT)

   An Acceptance Plan includes more details, not necessarily pertaining to system functionality, but more of the test plan approach. Possible questions may include:
   - Are any special tools to be used and what are they?
   - Will the tool require special training?
   - What metrics will be collected?
   - Which level is each metric to be collected at?
   - How will maintenance to be handled?
   - How many different configurations will be tested?
   - Hardware requirements?
   - Software requirements?
   - Combinations of HW, SW and other vendor packages?
   - What levels of regression testing will be done and how much at each test level?
   - Will regression testing be based on severity of defects detected?
   - How will elements in the requirements and design that do not make sense or are untestable be processed?

   You need to specify if there are special requirements for the testing such as if only the full component will be tested or a specified segment of grouping of features/components must be tested together.

2. Test Plan
   2.1. Features to be tested

This is a listing of what is to be tested from the USERS viewpoint of what the system does. Also called a User Acceptance Test (UAT), this is not a technical description of the software, but a USERS view of the functions.

It should be noted that Section 1.1 and Section 2.1 are very similar. The only true difference is the point of view. Section 1.1 is a technical type description including version numbers and other technical information and Section 2.1 is from the User's viewpoint. Users do not understand technical software terminology; they understand functions and processes as they relate to their jobs. It is possible to have overlapping test cases for Section 1.1 and 2.1.

2.2. Item Pass/Fail Criteria

To assess if the test plan has passed or failed, the decision would have to come from the developer's perspective in accordance with the system requirements and the test plan method used. This is a critical aspect of any test plan and should be appropriate to the level of the plan.

At the Master test plan level, this could be items such as a specified number of plans completed without errors and a percentage with minor defects. This could be an individual test case level criterion or a unit level plan or it can be general functional requirements for higher level plans.

At the Unit test level this could be items such as:

- All test cases completed.
- A specified percentage of cases completed with a percentage containing some number of minor defects.
- Code coverage tool indicates all code covered.

2.3. Test deliverables

The following needs to be included in the final document to be submitted in SE2 (Software Engineering 2):

- Software Test Document
- Test design specifications
- Tools and their output
- Test cases *(see sample template on Appendix E)*

Test cases need to be documented at all times, including failures/errors. Date and time logs are important to keep track of system integrity.

APPENDIX E
Test Case Template

Project Name:

Test Case Template

Test Case ID: Fun_10	**Test Designed by:** <Name>
Test Priority (Low/Medium/High): Med	**Test Designed date:** <Date>
Module Name: Google login screen	**Test Executed by:** <Name>
Test Title: Verify login with valid username and password	**Test Execution date:** <Date>
Description: Test the Google login page	

Pre-conditions: User has valid username and password
Dependencies:

Step	Test Steps	Test Data	Expected Result	Actual Result	Status (Pass/Fail)	Notes
1	Navigate to login page	User: example@gmail.com	User should be able to login	User is navigated to	Pass	
2	Provide valid username	Password: 1234		dashboard with successful		
3	Provide valid password			login		
4	Click on Login button					

Post-conditions:
User is validated with database and successfully login to account. The account session details are logged in database.

APPENDIX F
List of Final Project Deliverables

Below is the checklist for the final project deliverables for **Software Engineering 2**:

Major Requirements

1. Signed Sign-off sheet
2. Accomplished UAT checklist (System Testing)
3. Sample project output reports/screenshots
4. Accomplished Survey (UAT) results (during Beta Testing)

Minor Requirements

1. Photo proof of client signing & answering the Sign-off sheet and UAT checklist
2. Computations:
    a. Function Point (FP)
    b. Software Maturity Index (SMI)
    c. Defect Ratio Efficiency (DRE)
3. User's Manual, Installation Manual, and/or Instruction Manual
4. Full document in a single PDF file (SPMP, SRS, SDD, STD, applicable Manuals)
5. Full system source code in a ZIP file

APPENDIX G
Letter Formats

Proposal Letter

Request for Interview

UAT Request Letter

SPMP cover letter

APPENDIX H
Evaluation Sheets

Software Engineering 1 (SE1) Evaluation Sheet

Software Engineering 2 (SE2) Evaluation Sheet

APPENDIX I
Project Forms

For SE1

Project Development form (SE1)

Certificate of Non-Relation

For SE2

Project Deployment form (SE2)

SE2 checklist

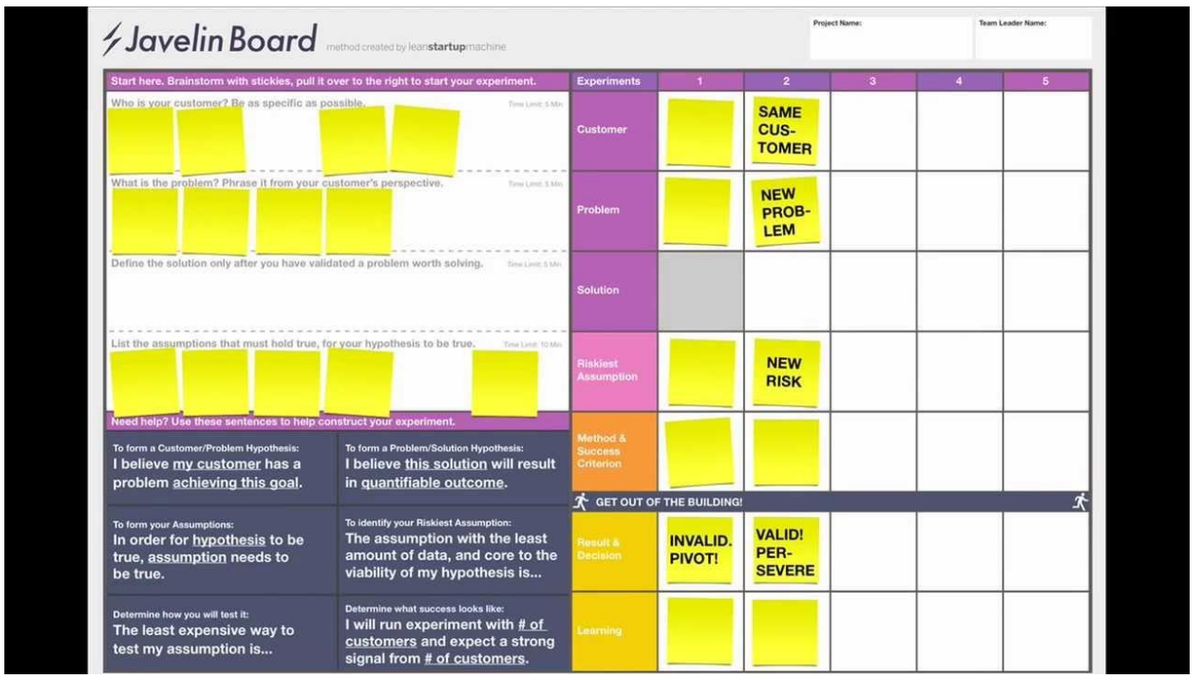
UAT checklist

Sign Off Sheet

APPENDIX J
VALIDATION BOARD

The validation board can be downloaded from https://www.leanstartupmachine.com/validationboard/

The site includes video tutorials on how to create the validation board as well as printing instructions.



References

Business Analyst. (2018, July 14). Retrieved from Wikipedia: https://en.wikipedia.org/wiki/Business_analyst

IEEE Software Requirements Specification Template. (n.d.). Retrieved from https://web.cs.dal.ca/~hawkey/3130/srs_template-ieee.doc

IEEE Standard for Software Project Management Plans. (n.d.). Retrieved from Middle East Technological University: https://cow.ceng.metu.edu.tr/Courses/download_courseFile.php?id=4047

Object Oriented Systems Analysis and Design. (2018, March 30). Retrieved from W3Computing: http://www.w3computing.com/systemsanalysis/object-oriented-systems-analysis-design/

Project Based Learning. (2018, March 30). Retrieved from Wikipedia: https://en.wikipedia.org/wiki/Project-based_learning

Project Manager. (2018, May 10). Retrieved from Wikipedia: https://en.wikipedia.org/wiki/Project_manager#Software_Project_Manager/IT_Project_Manager

Sample Test Case Template with Test Case Examples. (2018, June 7). Retrieved from Software Testing help: https://www.softwaretestinghelp.com/test-case-template-examples/

Software Design Document Template. (n.d.). Retrieved from https://sovannarith.files.wordpress.com/2012/07/sdd_template.pdf

Software Developer. (2018, July 17). Retrieved from Wikipedia: https://en.wikipedia.org/wiki/Software_developer

System Analyst. (2018, June 22). Retrieved from Wikipedia: https://en.wikipedia.org/wiki/Systems_analyst

Test Case Specification Template (IEEE 829-1998) . (n.d.). Retrieved from http://www.ufjf.br/eduardo_barrere/files/2011/06/SQETestCaseSpecificationTemplate.pdf

Test Plan Template (IEEE 829-1998 Format). (n.d.). Retrieved from http://www.ecs.csun.edu/~rlingard/comp480/TestPlanTemplate.pdf

The Systems Development Life Cycle. (2018, March 30). Retrieved from W3Computing: http://www.w3computing.com/systemsanalysis/systems-development-life-cycle/

The Ultimate Guide.. Waterfall Model. (2021, August 10). Retrieved from ProjectManager: https://www.projectmanager.com/waterfall-methodology

What does a QA Analyst Do? (2017, December 18). Retrieved from Rasmussen College: https://www.rasmussen.edu/degrees/technology/blog/what-does-a-qa-analyst-do/

What is Project Based Learning (PBL)? (2018, March 30). Retrieved from bie.org: https://www.bie.org/about/what_pbl

What is the Difference Between Software Engineering and Software? (2018, March 30). Retrieved from Computer Science Degree Hub: https://www.computersciencedegreehub.com/faq/what-is-the-difference-between-software-engineering-and-software/